

Professional Training Guide

JavaScript • TypeScript • Google Apps Script

Part 1 (Topics 1–10)

Training Edition • Beginner Friendly • Team Ready

These 20 topics are the common foundation used across all three technologies because Google Apps Script uses JavaScript, and TypeScript is an advanced version of JavaScript.

Page1: Topic 1 — Variables & Data Types

What Are Variables?

Variables are containers used to store data.

Or Variables are containers Which Conatin Data.

Why Variables Are Important

Everything in programming uses variables.

User names

Email addresses

Numbers and calculations

Employee CardNo.

How many Types of Variables

let - let city = "Bangalore";

const - let marks = 80;

var (old way) - var ignores block scope. Even outside the block, x still exists.

How Many Data Types

Primitive Data Types

Non-Primitive (Reference) Data Types

Primitive Data Types

These store single values. There are 7 primitive data types in JavaScript.

2. Non-Primitive (Reference) Data Types

These store collections or complex data.

Real Project Example: TypeScript

```
const employee = {  
  name: "Akhil",  
  age: 25,  
  skills: ["JS", "TS", "GAS"],  
  isActive: true  
};
```

Golden Rule

Use const first.

Use let only if needed.

Avoid var.

Page2: Topic 2 — Functions

What Is a Function?

Function In the sense Method

Why We use

Using function We can avoid repeating code.

or you can say A function is a reusable block of code.

Example1:

```
function add(a, b) {  
  console.log(a + b);  
}  
add(10, 20);
```

Example2:

```
// function
```

```
function greet(name) {  
  
  return "Hello " + name;  
}
```

```
console.log(greet("Akhil"));
```

```
// arrow function  
const add = (a, b) => a + b;  
console.log(add(2, 3));
```

Types of Functions

Main function types:

Normal Function

Function Expression

Arrow Function

Anonymous Function

Callback Function

Most Important Functions to Learn First

Function Declaration

Parameters

Return

Arrow Functions

Callback Functions

Array Methods with Functions

Async Functions

Golden Rule: Functions help write reusable, clean, and organized code.

Page3: Topic 3 — Objects

An object is used to store multiple related values in one place.

or Objects store related data together.

Objects store data in: 'key : value' format.

Simple Object Example

```
const student = {  
  name: "Akhil",  
  age: 25,
```

```
    city: "Bangalore"  
};
```

Here:

Objects can contain:

Strings

Numbers

Booleans

Arrays

Other objects

Functions

There are mainly 5 types of objects:

User-Defined Objects

Function Objects

Constructor/Class Objects

Environment Objects

Built-in Objects

Golden Rule

Objects are used to group related data together.

They are one of the most important concepts in JavaScript, TypeScript, and Google Apps Script.

Page4: Topic 4 — Arrays

What Is an Array?

Arrays is container which contains similar data.

Or

Arrays store multiple values in one variable.

Why it's used?

Instead of creating many variables:

```
let fruit1 = "Apple";
```

```
let fruit2 = "Banana";
```

```
let fruit3 = "Mango";
```

We use an array:

```
let fruits = ["Apple", "Banana", "Mango"];
```

Example

```
const fruits = ["Apple", "Banana", "Mango"];
```

Common Array Methods

`push()`

Add item.

```
fruits.push("Orange");
```

`pop()`

Remove last item.

```
fruits.pop();
```

Loop Through Arrays

```
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

Common Array Methods

Golden Rule

Arrays are used to store and manage multiple values efficiently.

They are one of the most important concepts in JavaScript, TypeScript, and Google Apps Script.

Page5: Topic 5 — Conditionals

What Are Conditionals?

Conditionals help programs make decisions.

Conditionals are used to make decisions in code.

Real Example:

If age is 18 or more → Allow voting

Else → Do not allow

Main Conditional Statements

Comparison Operators

Used in conditions.

Example

```
const age = 20;
```

```
// if else
```

```
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

```
// ternary
```

```
console.log(age >= 18 ? "Adult" : "Minor");
```

```
// switch
```

```
switch(age) {  
  
  case 20:  
  
    console.log("Age is 20");  
}
```

```
break;
```

```
}
```

```
// AND &&
```

```
if (age >= 18 && true) {
```

```
  console.log("Allowed");
```

```
}
```

```
// optional chaining
```

```
const user = {};
```

```
console.log(user.address?.city);
```

```
// nullish ??
```

```
console.log(user.name ?? "Guest");
```

Golden Rule

Conditionals help programs make decisions.

They are essential for building logic in JavaScript, TypeScript, and Google Apps Script.

Page6: Topic 6 — Conditionals

Loops repeat code automatically.

In Loop we also provide condition

Loops are used to repeat code multiple times automatically.

Instead of writing:

```
console.log(1);
```

```
console.log(2);
```

```
console.log(3);
```

We use loops.

Most Important Loop Topics

for loop

while loop

do...while

for...of

for...in

forEach

break

continue

nested loops

array methods

Example

// for

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

// while

```
let n = 0;
```

```
while (n < 3) {  
  console.log(n);  
  n++;  
}
```

// for...of

```
for (const x of ["A", "B"]) {  
  console.log(x);  
}
```

Golden Rule

Loops help automate repetitive tasks efficiently.

They are essential for working with arrays, objects, and data in JavaScript, TypeScript, and Google Apps Script.

Page7: DOM & Browser Basics

What Is DOM?

DOM means Document Object Model.

It allows JavaScript to interact with webpages.

or

It is how JavaScript interacts with a webpage.

When a browser loads HTML, it converts the page into a tree-like structure called the DOM.

JavaScript can then:

Read elements

Change content

Change styles

Handle button clicks

Create/remove elements

Simple Understanding

HTML creates the webpage.

JavaScript controls the webpage.

Example:

```
<h1 id="title">Hello</h1>
```

JavaScript:

```
document.getElementById("title").textContent = "Welcome";
```

Result on webpage:

Welcome

Example:

```
// select element
```

```
const box = document.getElementById("box");
```

```
// change content & style
```

```
box.innerText = "Hello";
```

```
box.style.color = "blue";
```

```
// click event
```

```
box.addEventListener("click", () => {
```

```
  alert("Clicked");
```

```
});
```

```
// create element
```

```
const p = document.createElement("p");
```

```
p.innerText = "New Paragraph";
```

```
document.body.appendChild(p);
```

```
// browser basics
```

```
console.log(window.innerWidth);
```

```
console.log(location.href);
```

```
console.log(navigator.userAgent);
```

```
// timer
```

```
setTimeout(() => {
```

```
  console.log("Done");
```

```
}, 1000);
```

Golden Rule

DOM allows JavaScript to control and update webpages dynamically.

It is one of the most important topics for frontend web development.

Page8: Topic 8 — Async JavaScript

What Is Async JavaScript?

Async code allows tasks to run without freezing the application.

or

Async means:

Running tasks without blocking other code.

JavaScript is single-threaded, but async features allow it to handle:

API calls

Timers

File loading

Database requests

User actions

without freezing the program.

Synchronous vs Asynchronous

Synchronous

Runs line by line.

```
console.log("1");
```

```
console.log("2");
```

```
console.log("3");
```

Output:

1

2

3

Asynchronous

Some tasks run later.

```
console.log("1");

setTimeout(() => {

    console.log("2");

}, 2000);

console.log("3");
```

Output:

```
1
3
2
```

Because `setTimeout()` runs later.

Why Async JavaScript Is Important

Used for:

Fetching APIs

Waiting for server response

Loading files

Timers

Real-time apps

Modern web apps depend heavily on async code.

Example:

// callback

```
setTimeout(() => {

    console.log("Callback");

}, 1000);
```

// promise

```
const data = new Promise((resolve) => {

    resolve("Promise Done");

});
```

```
data.then(res => console.log(res));
```

```
// async/await  
  
async function getData() {  
  return "Async Done";  
}  
  
getData().then(res => console.log(res));
```

Golden Rule

Async JavaScript allows programs to handle waiting tasks efficiently without blocking execution.

It is one of the most important modern JavaScript concepts.

Page9: Topic 9 — ES6+ Modern JavaScript

What Is ES6?

ES6 introduced modern JavaScript features.

ES6 means:

ECMAScript 2015

It introduced modern JavaScript features.

After ES6, many new versions came:

ES7

ES8

ES9

ES10+

Modern JavaScript (ES6+)

Why ES6+ Is Important

Modern JavaScript makes code:

Shorter

Cleaner

Faster

Easier to read

Easier to maintain

Most modern projects use ES6+ syntax.

Most Important ES6+ Features

Example:

```
// let & const
const name = "Akhil";

// template literal
console.log(`Hello ${name}`);

// arrow function
const add = (a, b) => a + b;

// destructuring
const user = { age: 20 };
const { age } = user;

// spread
const arr = [1, 2];
console.log([...arr, 3]);

// default parameter
function greet(msg = "Hi") {
  console.log(msg);
}

// optional chaining
console.log(user.address?.city);

// nullish ??
console.log(user.city ?? "Bangalore");
```

Golden Rule

ES6+ introduced modern, cleaner, and powerful JavaScript features.

Modern JavaScript development heavily depends on ES6+ concepts.

Page10: Topic 10 — Error Handling

Why Error Handling Matters

Programs can fail.

Error handling prevents application crashes.

or

Without error handling, applications can stop unexpectedly.

What Is an Error?

An error happens when code fails.

Example:

```
console.log(userName);
```

Output:

ReferenceError

Because userName is not defined.

Why Error Handling Is Important

Helps to:

Avoid crashes

Debug problems

Handle API failures

Handle invalid input

Build stable applications

Main Error Handling Concepts

Example:

```
// try catch
```

```
try {  
  console.log(a);  
} catch (err) {  
  console.log("Error");  
} finally {  
  console.log("Done");  
}
```

```
// throw

function check(age) {

  if (age < 18) {
    throw "Not Allowed";
  }

  return "Allowed";
}

console.log(check(20));
```

Golden Rule

Error handling helps applications fail safely instead of crashing.

It is essential for writing reliable JavaScript, TypeScript, and Google Apps Script applications.